

Cleanverse Compliance Protocol (CCP)

Integration Guide (For CVI Compliance Validator) V2

Overview

The CVI Compliance Validator (IAPassComplianceValidator) provides on-chain identity compliance verification based on CVI (Cleanverse Verified Identity) for DeFi protocols. This guide is intended for protocol developers and explains how to integrate the validator to implement KYC / compliance gates.

What the Validator Does

- Verify whether a user's CVI satisfies the compliance rules configured for a pool (Group / Tier / Sub-Group / Sub-Tier / country bitmap)
- Manage per-pool compliance rules (multiple rules per pool, OR logic)
- Register CVI for CVA (Cleanverse Verified Asset) vaults (Pool + Fee) so they can hold / transfer CVAs
- Pause pools or freeze accounts (emergency risk control)

Choosing an Integration Mode

Pick the integration mode based on business complexity:

Scenario	Mode	Notes
Single business contract (lending, staking, NFT)	Single-contract mode	Local deployment, no Factory authorization required
Multi-pool business (DEX, Launch Pool)	Factory mode	One authorization, batch management of multiple pools

Core Interface Specification

3.1 RuleV2 Data Structure

Rule struct

```

1 struct RuleV2 {
2     bytes2 allowedGroup;           // Allowed CVI group (empty = no restriction)
3     bytes2 allowedSubGroup;       // Allowed CVI sub-group (empty = no
restriction)
4     uint8 minTier;                // Minimum CVI tier (0 = no restriction)
5     uint8 minSubTier;             // Minimum CVI sub-tier (0 = no restriction)
6     uint256 poolCountryBitmap;    // Country bitmap (0 = no restriction)
7 }

```

Validation Logic: Fields within a single RuleV2 are AND; multiple RuleV2s are OR; country bitmaps are checked via bitwise AND.

3.2 Interface List

Registration (REGISTER_ROLE)

IAPassComplianceValidator Interface

```

1 function registerV2(address poolAddress, RuleV2 calldata rule) external;
2 function registerApass(address poolAddress, address aTokenAddress) external;
3 function registerApass(address poolAddress, address aTokenAddress, address
feeAddress) external;
4 function setRuleV2FromRegistrar(address poolAddress, RuleV2 calldata rule)
external;
5 function isRegistered(address poolAddress) external view returns (bool);

```

Rule Management (Business Contract Itself)

IAPassComplianceValidator Interface

```

1 function setRuleV2FromContract(RuleV2 calldata rule) external;
2 function addRuleV2FromContract(RuleV2 calldata rule) external;
3 function removeRuleV2FromContract(uint256 index) external;
4 function getRulesV2(address poolAddress) external view returns (RuleV2[]
memory);

```

Compliance Verification (No Permission Required)

IAPassComplianceValidator Interface

```

1 function complianceVerify(address poolAddress, address userAddress) external
view returns (bool);

```

Pattern 1: Factory Mode

4.1 Use Cases

Factory mode is designed for multi-pool businesses (DEX, Launch Pool). Once the Factory holds the `REGISTER_ROLE`, it can call `registerV2` / `registerApass` directly when creating pools.

Typical Scenarios:

- DEX pools: BTC/USDT pool requires Tier 30, ETH/USDT pool requires Tier 40, institutional trading pairs require Group "ab" + specific country bitmap
- Launch Pool: create a separate pool for each new project with differentiated access thresholds

Core Advantages:

- One authorization, batch-manage multiple pools
- Each pool has independent compliance policy, no cross-impact
- Supports CVA Pool + Fee dual-address registration

4.2 Authorization Flow

The Factory must hold the `REGISTER_ROLE` to call `registerV2` / `registerApass`.

Workflow:

workflow

- 1 Step 1: Deploy the Factory contract
- 2 ↓
- 3 Step 2: Call the API to authorize the Factory address (grant `REGISTER_ROLE`)
- 4 ↓
- 5 Step 3: Factory calls `registerV2` / `registerApass` to register pools
- 6 ↓
- 7 Step 4: Pool business contract calls `complianceVerify` for compliance checks

Authorization API: `POST /api/cooperate/validator/grant`

4.3 Factory Contract Template

The Factory must implement:

- Inherit `Ownable`, only Owner can create pools

- Store the validator address (immutable)
- Call the validator interface to register pools and manage rules

Contract Template:

Factory Contract Template

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.24;
3
4  import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
5  import {IAPassComplianceValidator} from "../IAPassComplianceValidator.sol";
6
7  /**
8   * @title DexLaunchFactory
9   * @notice DEX liquidity pool factory with CVI V2 compliance support
10  */
11  contract DexLaunchFactory is Ownable {
12
13      address public immutable complianceValidator;
14
15      /// @notice Pool information
16      struct PoolInfo {
17          address pool;
18          address aToken;
19          address feeAccount;
20          bool isATokenPool;
21          uint256 createdAt;
22      }
23
24      PoolInfo[] public pools;
25
26      constructor(address validator_, address owner_) {
27          require(validator_ != address(0), "validator=0");
28          complianceValidator = validator_;
29          _transferOwnership(owner_);
30      }
31
32      /// @notice Create a regular V2 pool (uses validator directly)
33      /// @param poolAddress Pool contract address
34      /// @param rule V2 compliance rule
35      function createPoolV2(
36          address poolAddress,
37          IAPassComplianceValidator.RuleV2 calldata rule
38      ) external onlyOwner {
39          IAPassComplianceValidator(complianceValidator).registerV2(poolAddress,
40 rule);

```

```

40     pools.push(PoolInfo({
41         pool: poolAddress,
42         aToken: address(0),
43         feeAccount: address(0),
44         isATokenPool: false,
45         createdAt: block.timestamp
46     }));
47 }
48
49 /// @notice Create a CVA pool (registers Pool + Fee together)
50 /// @param poolAddress Pool contract address
51 /// @param aTokenAddress CVA address
52 /// @param feeAddress Fee account address
53 /// @param rule V2 compliance rule
54 function createATokenPoolV2(
55     address poolAddress,
56     address aTokenAddress,
57     address feeAddress,
58     IAPassComplianceValidator.RuleV2 calldata rule
59 ) external onlyOwner {
60     // 1. Register pool and set V2 compliance rule
61     IAPassComplianceValidator(complianceValidator).registerV2(poolAddress,
rule);
62     // 2. Register CVI for Pool + Fee addresses
63     IAPassComplianceValidator(complianceValidator).registerApass(
64         poolAddress, aTokenAddress, feeAddress
65     );
66     pools.push(PoolInfo({
67         pool: poolAddress,
68         aToken: aTokenAddress,
69         feeAccount: feeAddress,
70         isATokenPool: true,
71         createdAt: block.timestamp
72     }));
73 }
74 }

```

4.4 Pool Contract Template

Each Pool contract must implement:

- Store the validator address (immutable)
- Call `complianceVerify` at key business steps
- Optionally store the Factory address for caller verification

Contract Template:

Pool Contract Template

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.24;
3
4  import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
5  import {IAPassComplianceValidator} from "../IAPassComplianceValidator.sol";
6
7  /**
8   * @title CompliantDexPool
9   * @notice DEX pool with CVI V2 compliance support
10  */
11  contract CompliantDexPool is Ownable {
12
13      IAPassComplianceValidator public immutable validator;
14      address public immutable factory;
15
16      // Trading pair info
17      address public token0;
18      address public token1;
19
20      constructor(address validator_, address factory_) {
21          require(validator_ != address(0), "validator=0");
22          validator = IAPassComplianceValidator(validator_);
23          factory = factory_;
24      }
25
26      /// @notice Verify CVI when adding liquidity
27      /// @param to Recipient of the liquidity share
28      /// @param amount Liquidity amount
29      function addLiquidity(address to, uint256 amount) external {
30          // Verify the recipient meets compliance requirements
31          require(
32              validator.complianceVerify(address(this), to),
33              "A-Pass not qualified"
34          );
35          // Add liquidity logic...
36      }
37
38      /// @notice Verify CVI on swap
39      /// @param from Sender
40      /// @param to Recipient
41      /// @param amount Swap amount
42      function swap(address from, address to, uint256 amount) external {
43          // Verify both sides meet compliance requirements
44          require(
45              validator.complianceVerify(address(this), from) &&
```

```

46     validator.complianceVerify(address(this), to),
47     "A-Pass not qualified"
48 );
49     // Swap logic...
50 }
51
52 // ——— V2 Rule Management

```

```

53
54     /// @notice Set V2 rule (clears existing V2 rules first, then sets the new
one)
55     function setRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata
rule) external onlyOwner {
56         validator.setRuleV2FromContract(rule);
57     }
58
59     /// @notice Append a V2 rule (OR logic)
60     function addRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata
rule) external onlyOwner {
61         validator.addRuleV2FromContract(rule);
62     }
63
64     /// @notice Remove a V2 rule by index
65     function removeRuleV2FromContract(uint256 index) external onlyOwner {
66         validator.removeRuleV2FromContract(index);
67     }
68
69     /// @notice Get current V2 rules
70     function getRulesV2() external view returns
(IAPassComplianceValidator.RuleV2[] memory) {
71         return validator.getRulesV2(address(this));
72     }
73 }

```

4.5 Integration Method A: Using CVA (Automatic Compliance)

When using CVA (AUSDT / AUSDC), compliance checks are performed automatically by the CVA contract — the business contract does not need to call the validator explicitly. After registering the pool, simply call `registerApass` to issue CVI for the Pool (+ Fee) address.

Use Cases:

- The platform already has CVA as a trading pair
- The protocol needs a Fee account to collect fees (use `registerApass(pool, token, fee)`)

Notes:

- `registerApass` can only be called by the Factory, and only for CVA vault registration
- A fee address of `address(0)` skips the Fee CVI registration

Integration Steps:

Step Workflow

- 1 Step 1: Factory calls `registerV2` to register the pool and set rules
- 2 ↓
- 3 Step 2: Factory calls `registerApass(pool, token, fee)` to register CVI
- 4 ↓
- 5 Step 3: Users deposit CVA into the vault
- 6 ↓
- 7 Step 4: On transfers, the CVA contract automatically verifies CVI

Key Code:

Pool Key Code


```

1 // Factory creates a CVA pool
2 function createATokenPoolV2(
3     address poolAddress,
4     address aTokenAddress,
5     address feeAddress,
6     IAPassComplianceValidator.RuleV2 calldata rule
7 ) external onlyOwner {
8     // 1. Register pool and set V2 compliance rule
9     IAPassComplianceValidator(validator).registerV2(poolAddress, rule);
10    // 2. Register CVI for Pool + Fee addresses
11    IAPassComplianceValidator(validator).registerApass(
12        poolAddress, aTokenAddress, feeAddress
13    );
14 }
15
16 // CVA contract performs automatic compliance checks (no explicit validator
17 // call from business contract)
18 function _update(address from, address to, uint256 amount) internal override {
19     // Both sides must pass CVI verification
20     if (!validator.complianceVerify(address(this), from)) {
21         revert TransferNotAllowed();
22     }
23     if (!validator.complianceVerify(address(this), to)) {
24         revert TransferNotAllowed();
25     }
26     super._update(from, to, amount);
27 }

```

4.6 Integration Method B: Calling the Validator Directly

When CVA is not used, the business contract calls `complianceVerify` at key business steps.

Use Cases:

- Use native ERC20 as a trading pair
- Need additional compliance checks at the business layer
- Build more complex custom access logic

Integration Steps:

Step Workflow

- Step 1: Factory calls `registerV2` to register the pool
- ↓
- Step 2: User calls a business method
- ↓
- Step 3: Business contract calls `complianceVerify`

6 ↓
7 Step 4: Pass → business executes | Fail → revert

Key Code:

Pool Key Code

```
1  // Factory registers a V2 pool
2  function createPoolV2(
3      address poolAddress,
4      IAPassComplianceValidator.RuleV2 calldata rule
5  ) external onlyOwner {
6      IAPassComplianceValidator(poolAddress).registerV2(rule);
7  }
8
9  // Pool contract calls the validator in business logic
10 function swap(address from, address to, uint256 amount) external {
11     require(
12         validator.complianceVerify(address(this), from) &&
13         validator.complianceVerify(address(this), to),
14         "A-Pass not qualified"
15     );
16     // Swap logic...
17 }
```

Pattern 2: Single-Contract Mode

5.1 Use Cases

Single-contract mode does not require Factory authorization. Deploy the contract and register it via the API to start using the validator. Suitable for:

- Lending protocols: verify borrower CVI to filter compliant borrowers
- NFT minting: whitelist minting, prioritize community contributors
- Staking pools: high-yield pools restrict high-tier users
- Governance voting: limit votes to qualified participants

5.2 Contract Template

The business contract must implement:

- Store the validator address (immutable)

- Call `complianceVerify` at key business steps
- Inherit `Ownable` to protect management functions

Contract Template:

Single Contract Template

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.24;
3
4  import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
5  import {IAPassComplianceValidator} from "../IAPassComplianceValidator.sol";
6
7  /**
8   * @title CompliantLending
9   * @notice Lending protocol with CVI V2 compliance support
10  */
11  contract CompliantLending is Ownable {
12
13      IAPassComplianceValidator public immutable validator;
14
15      mapping(address => uint256) public deposits;
16      mapping(address => uint256) public borrowings;
17
18      constructor(address validator_) {
19          require(validator_ != address(0), "validator=0");
20          validator = IAPassComplianceValidator(validator_);
21      }
22
23      /// @notice Deposit
24      function deposit(uint256 amount) external {
25          // Verify the depositor's CVI
26          require(
27              validator.complianceVerify(address(this), msg.sender),
28              "A-Pass not qualified"
29          );
30          deposits[msg.sender] += amount;
31      }
32
33      /// @notice Borrow
34      function borrow(uint256 amount) external {
35          // Verify the borrower's CVI (can set higher thresholds)
36          require(
37              validator.complianceVerify(address(this), msg.sender),
38              "A-Pass not qualified"
39          );
40          borrowings[msg.sender] += amount;

```

```

41     }
42
43     /// @notice Withdraw
44     function withdraw(uint256 amount) external {
45         // Verify the withdrawer's CVI
46         require(
47             validator.complianceVerify(address(this), msg.sender),
48             "A-Pass not qualified"
49         );
50         require(deposits[msg.sender] >= amount, "insufficient balance");
51         deposits[msg.sender] -= amount;
52     }
53
54     /// — V2 Rule Management
55
56     /// @notice Set V2 rule (clears existing V2 rules first, then sets the new
one)
57     function setRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata
rule) external onlyOwner {
58         validator.setRuleV2FromContract(rule);
59     }
60
61     /// @notice Append a V2 rule (OR logic)
62     function addRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata
rule) external onlyOwner {
63         validator.addRuleV2FromContract(rule);
64     }
65
66     /// @notice Remove a V2 rule by index
67     function removeRuleV2FromContract(uint256 index) external onlyOwner {
68         validator.removeRuleV2FromContract(index);
69     }
70
71     /// @notice Get current V2 rules
72     function getRulesV2() external view returns
(IAPassComplianceValidator.RuleV2[] memory) {
73         return validator.getRulesV2(address(this));
74     }
75 }

```

5.3 Integration Steps

Step Workflow

- 1 Step 1: Deploy the business contract

```
2      ↓
3  Step 2: Call the API to register the contract with the validator
4      ↓
5  Step 3: The operator sets rules via setRuleV2FromContract
6      ↓
7  Step 4: User calls a business method
8      ↓
9  Step 5: The contract internally calls complianceVerify
10     ↓
11  Step 6: Pass → business continues | Fail → revert
```

5.4 API Registration

Endpoint: POST /api/cooperate/validator/register

Signature Rule: keccak256(chain + contract_address), lowercase hex concatenation

Note: The API registration only binds the contract address; compliance checks are performed by the business contract via internal calls to the validator.

Compliance Rule Management

After a pool is registered, the business contract can manage compliance rules via the following methods:

Method	Behavior
setRuleV2FromContract(rule)	Replace all rules
addRuleV2FromContract(rule)	Append a rule (OR logic)
removeRuleV2FromContract(index)	Remove a rule by index
getRulesV2()	Query the rule list

Sample Code:

Rule Management Code

```
1  function setRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata rule)
   external onlyOwner {
2      validator.setRuleV2FromContract(rule);
3  }
4
5  function addRuleV2FromContract(IAPassComplianceValidator.RuleV2 calldata rule)
   external onlyOwner {
6      validator.addRuleV2FromContract(rule);
7  }
```

```
8
9  function removeRuleV2FromContract(uint256 index) external onlyOwner {
10      validator.removeRuleV2FromContract(index);
11  }
12
13  function getRulesV2() external view returns
14      (IAPassComplianceValidator.RuleV2[] memory) {
15      return validator.getRulesV2(address(this));
16  }
```

Access Control: The business contract should enforce `onlyOwner` or `AccessControl` on rule management methods.